PES University

Department of Computer Science and Engineering

Third Semester Computer Networks

Network Time Synchronization System

SDN Socket Programming Mini Project

Project 23 Team 11 Section K

Submitted by

Student name	SRN
SHREYANSH	PES2UG25CS734
VATSAL	PES2UG25CS709
ANNADATA VENKATA SHANMUKHA ANIKETH	PES2UG25CS714

Submitted to [Faculty guide name]

Academic year [Academic year]

Submission date [Submission date]

Repository Network-Time-Sync

This report documents the UDP application, Mininet network, SDN policy, recorded experiments, and final validation of the final branch. Student details are taken from the official team allocation workbook. [1, 2]

Contents

Abstract....3
Introduction and Problem Statement....4
Project Objectives....4
Technologies Used....5
System Architecture....6
Implementation....7
Client and Server Processing....8
SDN Forwarding and Quality of Service....9
Load Monitoring and Automatic Recovery....10
Experimental Setup and Methodology....11
Results and Performance Analysis....12
Performance Across Network Conditions....13
Failure Accounting and Recovery Evidence....14
Testing and Validation....15
Coverage of the Official Rubric....16
Challenges and Limitations....17
Future Scope....17
Conclusion....18
References....18
Appendix Setup and Execution....19

Report scope

The report follows the assigned Project 23 requirements. Architecture and implementation descriptions refer to the final branch; numerical comparisons refer to the preserved Linux VM archive. A separate final verification checks the current code without replacing the historical measurements.

Evidence convention

Bracketed numbers refer to the reference list. Run labels R1 to R5 follow timestamp order on 1 November 2026. Clock error is stated in milliseconds unless specified otherwise. Performance claims apply to this controlled experiment, not to arbitrary networks.

Abstract

Network Time Sync is a simplified UDP time-synchronization application integrated with an SDN-controlled Mininet network. Three clients maintain virtual clocks and exchange timestamps with one reference server. Each client estimates round-trip delay and clock offset, then corrects its virtual time. The host operating system clock is never changed.

The network has four Open vSwitch switches arranged as a diamond. Baseline forwarding keeps traffic on a 10 Mbps primary path. Optimized forwarding places synchronization traffic in a dedicated edge queue, limits background traffic, and moves synchronization requests and replies to a 50 Mbps alternate path when the measured primary-uplink load reaches 4 Mbps. Three consecutive low-load polls restore the original path.

Five completed 60-second-per-mode comparisons are preserved in the repository. Across these comparisons, optimized mean clock error ranges from 1.034 to 1.158 ms, compared with 53.329 to 60.927 ms in baseline mode. Mean RTT ranges from 21.959 to 22.892 ms in optimized mode and from 96.142 to 121.607 ms in baseline mode. The optimized runs contain 1,800 successful attempts out of 1,800; baseline contains 1,360 successes and 393 timeouts out of 1,753 attempts. [4]

A separate recovery experiment verifies alternate-path packets, deletion of temporary rules, increasing primary-path packet counters, and replies to all three clients after the load stops. Initial convergence is not uniformly faster: optimized convergence takes longer in three of the five comparisons. These observations support the combined queue-and-routing policy under the tested conditions, while leaving its individual contributions unresolved.

Table 1. Key facts verified from source and recorded run metadata

Item	Verified value
Application transport	UDP; custom 35-byte message
Network	6 hosts, 4 OpenFlow 1.3 switches, 2 core paths
Control policy	4 Mbps trigger; 2 s polling; 3 low-load polls for recovery
Evidence	5 completed comparisons and 1 recovery test
Scope	Virtual clocks inside one Linux VM; no OS clock discipline

Introduction and Problem Statement

Distributed applications often compare events recorded by different machines. Those comparisons become unreliable when the machines disagree about time. A client can run at a different clock rate from a reference server, and the network used to request the time adds delay of its own. The assigned problem is to implement UDP timestamp exchange for multiple clients and evaluate synchronization error under controlled network conditions. [1, 2]

Receiving the server time is not enough to establish the client time at receipt. A reply spends time in the network. The project therefore records four moments around each exchange: client transmission, server reception, server transmission, and client reception. These timestamps separate server processing time from the round trip and provide an estimate of the correction to apply.

The network introduces a second problem. When background traffic fills the primary path, the forward and reverse trips may have different delays. The offset calculation assumes approximately symmetric travel time; asymmetry can leave a residual clock error even when a response is valid. This motivates giving the synchronization flow different network treatment rather than changing the socket protocol.

The project compares two forwarding policies using the same clients and timestamp format. Baseline provides ordinary forwarding on Path A. Optimized mode combines edge queue treatment with alternate-path routing. The comparison evaluates the complete policy; it does not isolate the benefit of queue prioritization from the benefit of a higher-capacity path.

Project Objectives

- Implement UDP client and server components with an explicit, fixed-size packet format and request matching.
- Support three clients with controlled virtual-clock offset and drift, without changing system time.
- Calculate round-trip delay, estimate clock offset, and record synchronization attempts consistently.
- Deploy the application in a repeatable Mininet topology controlled through OpenFlow 1.3.
- Prioritize synchronization traffic and react to primary-path load through rerouting and automatic recovery.
- Compare baseline and optimized behavior under normal traffic, congestion, and added delay and jitter.
- Preserve measurements and flow evidence, and provide setup instructions, tests, and an explainable demo.

Technologies Used

The experiment runs inside a Linux VM because Mininet, network namespaces, Open vSwitch, and traffic-control interfaces depend on Linux facilities. Mininet runs real application processes and the Linux network stack while separating host network interfaces through namespaces. The emulated hosts still share the VM kernel and system clock. [3, 7]

Table 2. Verified software and its role

Technology	Version or interface	Role
Linux Mint	22.3; Ubuntu noble base	Recorded VM operating system
Python	3.12.3 in recorded VM	Sockets, clocks, worker processes, CLI, tests
Mininet	2.3.0-1.1	Six-host, four-switch network
Open vSwitch	3.3.9 package	OpenFlow forwarding and edge queues
OS-Ken	2.11.2	Controller application and event handlers
Eventlet	0.40.4	Controller runtime and cooperative monitoring
iperf	2.1.9 package; UDP mode	12M offered background traffic
Linux tc	HFSC and NetEm interfaces	Link shaping, controlled delay, and jitter
pandas / NumPy	3.1.0 / 2.5.4	CSV analysis and summary statistics
Matplotlib	3.11.2	Recorded-results plots
HTML CSS JavaScript	Local browser files	Clock controls and network explanation

The root timesync launcher creates or reuses an isolated Python environment, then opens the terminal menu. Setup requests sudo for network tools, starts Open vSwitch, and writes its dependency marker only after imports succeed. The five top-level Python packages are pinned in requirements.txt. This keeps application packages separate from the operating system Python.

OS-Ken is a maintained controller framework used through its event and OpenFlow message APIs. The project starts Eventlet before importing the manager. The recorded version above comes from the VM environment file, rather than the current online documentation version. [4, 8]

The browser files run locally and provide clock controls and a guided network view. The terminal benchmark is the source of the Mininet measurements. The evaluated load generator is iperf v2 in UDP mode; the separate background-traffic helper is not used to produce the archived benchmark results.

System Architecture

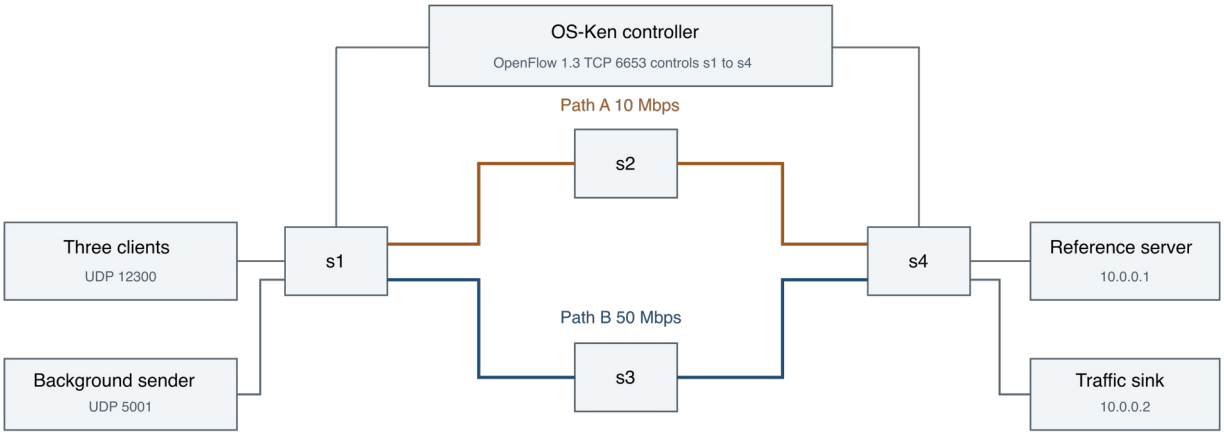


Figure 1. Four-switch diamond topology and separation of application and control traffic.

Three synchronization clients and the background sender attach to s1. The reference server and traffic sink attach to s4. Primary Path A crosses s2, while alternate Path B crosses s3. The controller programs forwarding rules; it does not receive and calculate each client clock correction. [3]

Table 3. Hosts and deterministic access ports

Host	IPv4 address	Access port	Function
h_client1	10.0.0.11	s1 port 1	Sync client; +20 ms/s drift
h_client2	10.0.0.12	s1 port 2	Sync client; -15 ms/s drift
h_client3	10.0.0.13	s1 port 3	Sync client; +30 ms/s drift
h_load	10.0.0.20	s1 port 4	Background UDP sender
h_server	10.0.0.1	s4 port 1	Reference time server
h_sink	10.0.0.2	s4 port 2	Background UDP receiver

The core links on Path A are shaped to 10 Mbps with 2 ms delay and queue size 100. Path B uses 50 Mbps, 2 ms delay, and queue size 1,000. Host access links use 100 Mbps and 1 ms delay. Fixed DPIDs and port numbers keep controller matches reproducible: s1 and s4 port 5 select A, and port 6 selects B.

Synchronization uses UDP 12300 and background traffic uses UDP 5001. TCP 6653 is the OpenFlow control channel, not the application transport. ARP and ICMP are infrastructure traffic used for address resolution and connectivity checks. This satisfies the Project 23 UDP requirement without treating every network packet as an application message. [1, 2]

Implementation

UDP Timestamp Exchange

app/protocol.py serializes requests and responses using !4sBHIddd. The exclamation mark selects network byte order and standard field sizes. Both message types have the same 35-byte layout. The VERSION constant exists in the module but is not serialized as a packet field. [3]

Table 4. Wire format shared by requests and replies

Field	Bytes	Meaning
Magic	4	NTPS identifies this application
Packet type	1	1 = request; 2 = response
Client ID	2	Unsigned client identifier
Sequence number	4	Unsigned attempt number
t0	8	Client virtual-clock send timestamp
t1	8	Server receive timestamp; zero in request
t2	8	Server reply timestamp; zero in request
Total	35	No t3 field; the client records t3 locally

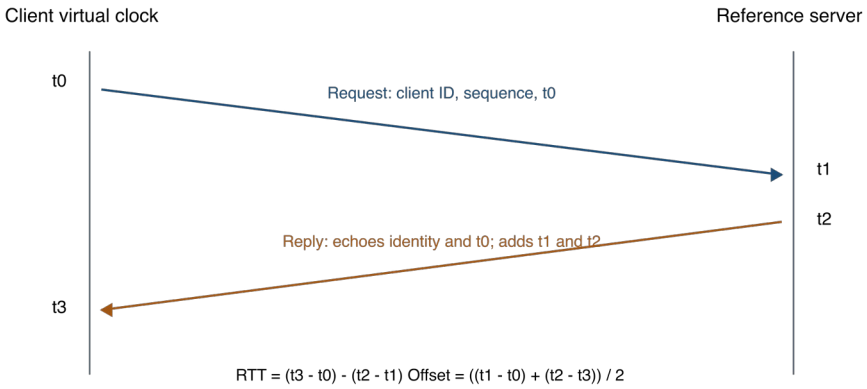


Figure 2. Four timestamps and the calculations used by both clients.

The server returns the client ID, sequence number, and original t0 unchanged, together with t1 and t2. The client records t3 immediately after receipt. Delay is expressed in seconds by the calculation functions and converted to milliseconds for reporting. Offset is the estimated amount to add to the client virtual clock.

These are the familiar four-timestamp delay and offset relationships described in time-synchronization literature. The custom packet is not wire-compatible with NTP, and the implementation does not claim to implement the complete NTP protocol or clock-selection algorithms. [5]

Client and Server Processing

Virtual Clock and Correction

VirtualClock computes time as system time plus an initial offset, accumulated artificial drift, and a correction term. The benchmark starts every client 0.100 s ahead of the reference. Its three drift rates are +0.020, -0.015, and +0.030 seconds per second. They are intentionally large so a short demonstration can show the effect of repeated correction.

```
virtual_time = real_time + initial_offset
               + elapsed_real * drift_rate + correction
```

Step adjustment adds the estimated offset immediately. The standalone client also supports slew adjustment, which applies a pending correction gradually, limited by max_slew_rate. The benchmark calls the step path. Neither method calls an operating-system clock-setting interface; the physical VM clock is unchanged.

Concurrency and Request Validation

The reference server binds a UDP socket, timestamps arrival, and submits each datagram to a ThreadPoolExecutor with four workers by default. A worker validates the packet, records the reply timestamp, sends the response, and logs the handled request. A receive timeout lets the main loop check its running flag. The socket and executor are closed in the shutdown path. Python provides the low-level socket operations used here. [3, 6]

A client accepts a response only when packet type, client ID, sequence number, and echoed t0 match the current request. Parsing rejects the wrong length, wrong magic, and non-finite timestamps. The arithmetic rejects impossible timestamp ordering, negative round-trip delay, and non-finite results. Clock correction happens only after these checks.

Table 5. Attempt outcomes and their accounting

Outcome	Meaning	Clock correction
SUCCESS	Matching, valid reply and valid timestamp calculation	Apply estimated offset
TIMEOUT	The socket wait expires or benchmark peer refuses the request	Do not adjust
INVALID	Reply decoding or request validation fails	Do not adjust

The standalone client records a CSV row when a reply is mismatched, rather than returning without a result. The benchmark also records one status per attempt. A rejected first reply ends that attempt; neither client implements a receive-until-deadline retry loop. A valid late reply can therefore be rejected by a later attempt, which is a stated limitation rather than guaranteed delivery.

SDN Forwarding and Quality of Service

The controller installs proactive rules when a switch connects. Baseline mode forwards client-to-server traffic across s1, s2, and s4 in the default queue. Optimized mode retains that base forwarding, adds application-specific matches, and prepares s3 for the alternate route. A table-miss rule sends otherwise unhandled packets to the controller. [3]

Table 6. Rule groups and priorities

Priority	Match or purpose	Action
0	Table miss	Send unhandled packet to controller
10	Base destination and core relay rules	Use deterministic Path A or host egress
50	Background port 5001	Use queue 2 at s1 Path A egress
100	UDP destination or source port 12300	Use normal sync forwarding; queue 1 on Path A edges
200	Temporary UDP 12300 reroute rules	Use s1-s3-s4 and the reverse route

Request rules match the synchronization destination port. Reply rules match the source port and, where needed, the client destination IP. These reverse-direction matches matter: moving requests alone would leave replies on the busy path and preserve timing asymmetry.

Table 7. HTB queues on s1-eth5 and s4-eth5

Queue	Minimum rate	Maximum rate	Role
0	1 Mbps	10 Mbps	Default traffic
1	2 Mbps	10 Mbps	Synchronization; priority value 1
2	1 Mbps	6 Mbps	Background load; priority value 10

Open vSwitch applies these linux-htb queues to egress on the two Path A edge ports. Queue 1 reserves bandwidth, and queue 2 caps the background sender. Queue IDs are meaningful on the configured egress port; the rerouted Path B rules use output actions rather than applying the Path A queue configuration throughout the network. OVS distinguishes egress shaping from ingress policing. [9]

Link shaping in the topology uses HFSC independently of the OVS HTB queues. The diamond also creates a potential broadcast loop. Instead of flooding ARP requests over both paths, the controller answers known host addresses from its deterministic host map. This supports address resolution while avoiding loop-driven ARP storms.

Load Monitoring and Automatic Recovery

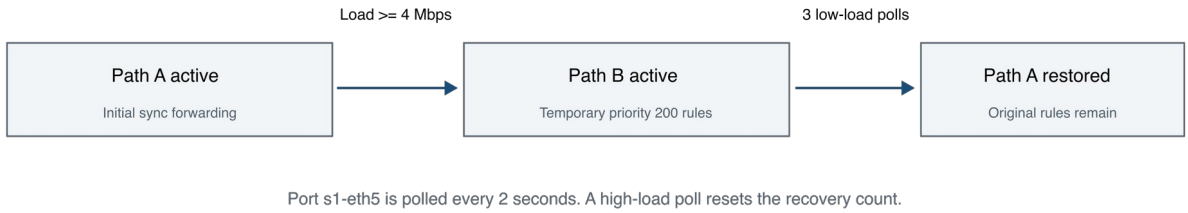


Figure 3. Load-triggered routing states and the three-poll recovery rule.

The optimized controller requests port statistics for switch s1 port 5 every two seconds. It stores transmit bytes, the polling timestamp, and dropped-packet counters. After a second observation, it estimates primary-uplink throughput from the byte and time differences. It logs dropped-counter differences but does not use drops as an independent trigger.

```
load_Mbps = 8 * (tx_bytes_now - tx_bytes_previous)
           / ((time_now - time_previous) * 1,000,000)
```

When throughput is at least 4 Mbps and rerouting is inactive, the controller installs priority-200 rules on s1, s3, and s4. Requests leave s1 through port 6 and cross s3 toward the reference server. Replies leave s4 through port 6 and return through s3. Background traffic keeps its Path A rules, so the alternate path is not given the same load source.

Priority determines which matching rule wins. The temporary 200 rules override the normal 100 sync rules, while base forwarding stays installed. A rule cookie is set from the priority. Recovery deletes only the temporary cookie group and matching sync traffic, allowing the original rules to become effective again.

A low-load poll increments a counter only while Path B is active. A high-load poll resets the counter. Three consecutive low-load polls restore Path A. With the configured poll interval this involves several seconds of observation, but the exact wall-clock return time depends on where load cessation falls between polls and on event scheduling. The report therefore does not infer a precise failover or recovery latency from the aggregate plots.

The controller observes network utilization, not the client clock error or a directly measured queue delay. Application identity influences which flow rules change; network port counters determine when they change. That relationship is the socket-SDN integration demonstrated by this project.

Experimental Setup and Methodology

The preserved archive contains five completed comparisons and one completed recovery run from 1 November 2026. Four incomplete attempts are excluded by their run.json status. The environment file identifies Linux Mint 22.3, Python 3.12.3, Mininet 2.3.0, Open vSwitch 3.3.9, OS-Ken 2.11.2, and Eventlet 0.40.4. The archived source commit is 06562b2; the implementation review and final tests use 55d1f57 on final. [3, 4]

Table 8. Common comparison settings

Setting	Value
Clients / initial offset	3 clients; all start +100 ms
Artificial drift	+20, -15, +30 ms per second
Sampling interval / socket timeout	0.5 s / 1.0 s in the benchmark
Mode duration	60 s baseline, then 60 s optimized
Background traffic	iperf v2 UDP 5001; offered rate 12M
Impairment point	Forward egress s2-eth2; 10 Mbps retained
Impairment parameters	50 ms delay and 15 ms jitter; queue size 100
Baseline / alternate core rates	10 Mbps / 50 Mbps

Table 9. Three phases within each comparison mode

Time	Condition	Purpose
0-20 s	Normal synchronization	Establish timing without background traffic
20-40 s	Add UDP background load	Observe contention and policy response
40-60 s	Retain load; add delay and jitter	Test sensitivity to an impaired forward path

Before sampling, the harness checks root privileges, required tools, controller imports, OVS readiness, and controller-port availability. It waits for the listening controller, builds and connects all switches, and probes every directed host pair. The six hosts produce 30 directed pairs. Startup and phase-end flow snapshots, queue records, and qdisc state are saved.

Workers use a shared monotonic start time, while protocol timestamps use virtual or reference wall time. Three persistent UDP sockets remain active throughout a mode. CSV files are flushed after each sample, and the run metadata is marked incomplete if execution raises an exception. Processes and network resources are stopped in the cleanup path.

Results and Performance Analysis

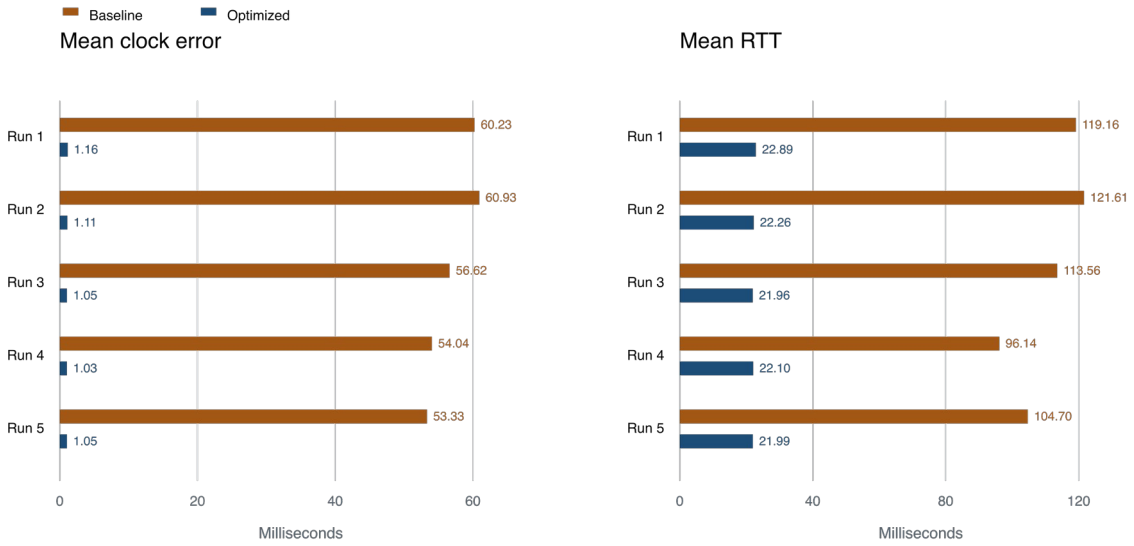


Figure 4. Per-run means recalculated from the archived client CSVs. R1-R5 follow timestamp order on 1 November 2026.

The optimized policy reduces mean clock error in every completed comparison, from baseline values of 53.329-60.927 ms to 1.034-1.158 ms. Mean RTT falls from 96.142-121.607 ms to 21.959-22.892 ms. The plotted values are per-run means; they are not confidence intervals or claims about every individual sample. [4]

Table 10. Failure percentage and initial convergence for each comparison

Run and time	Failed attempts B / O (%)	Convergence B / O (s)
R1 190534	19.075 / 0.000	1.503 / 3.502
R2 191520	20.506 / 0.000	1.503 / 5.502
R3 192057	20.613 / 0.000	4.504 / 1.504
R4 192459	28.783 / 0.000	8.006 / 3.502
R5 193256	23.380 / 0.000	1.503 / 3.502

B denotes baseline and O denotes optimized. Convergence is the confirmation time after three successive sequence rounds where every client has a successful reply and post-attempt error at or below 2 ms. It occurs later in optimized mode in R1, R2, and R5. These early-round results do not contradict the lower full-run error under congestion; they measure a different part of the run.

Performance Across Network Conditions

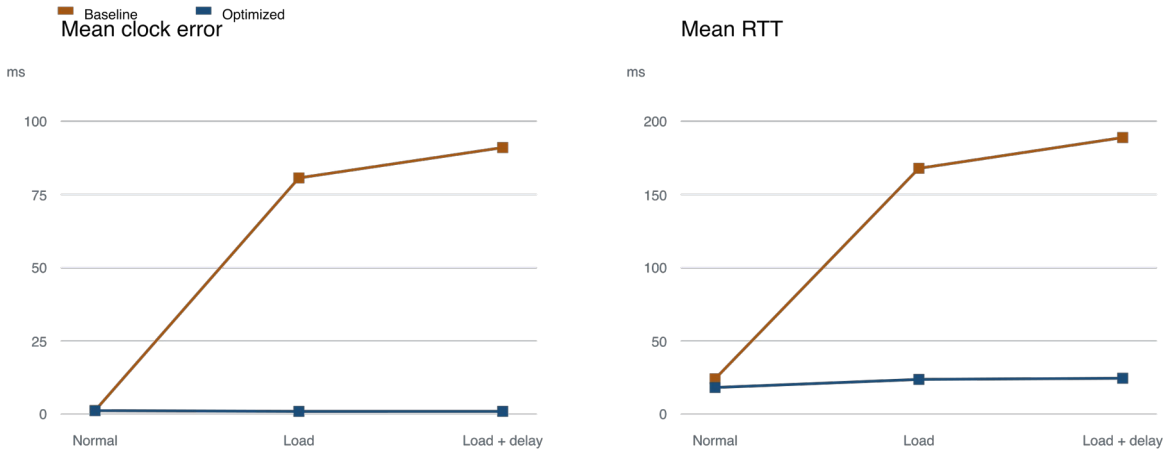


Figure 5. Mean of the five per-run phase means. RTT includes successful replies only; clock error includes every attempt.

Table 11. Clock error by phase averaged across the five run means

Phase	Baseline (ms)	Optimized (ms)
Normal	1.287	1.234
Background load	80.681	0.999
Load plus delay and jitter	91.045	1.007

Normal-phase errors are close in the two policies: approximately 1.287 ms baseline and 1.234 ms optimized. Once the background load starts, baseline mean error across the per-run phase means rises to 80.681 ms, while optimized remains near 0.999 ms. Adding forward delay and jitter raises baseline error to 91.045 ms; optimized remains near 1.007 ms. These values are derived from the raw phase-labelled CSVs in the preserved archive.

The result is consistent with the optimized path avoiding the impaired forward link and with synchronization traffic receiving different queue treatment. It cannot establish the separate contribution of either mechanism. Path B has five times the configured core bandwidth and does not carry the same background flow, so this is a comparison of two complete network policies, not a controlled isolation of one SDN feature.

RTT and jitter summaries use only successful responses. The jitter value is the absolute difference from the previous successful RTT within a client stream; it is reset after a failed attempt. Excluding missing replies means that successful-response timing must be interpreted alongside the failure counts on the next page.

Failure Accounting and Recovery Evidence

Table 12. Completed comparison attempts across all five runs

Mode	Attempts	Success	Timeout	Invalid
Baseline	1753	1360	393	0
Optimized	1800	1800	0	0

The client socket waits for at most one second per benchmark attempt. A timeout consumes wall-clock time, so a baseline run need not produce the same number of attempts as an optimized run at a nominal 0.5-second schedule. The baseline total contains 393 failed attempts; every recorded optimized attempt succeeds. These are application outcomes, not a measured packet-loss percentage. [4]

Mean clock error includes the absolute virtual-clock gap after a failed attempt, when no correction was applied. The common VM clock makes that gap observable. The values do not continuously measure error between attempts. The server-log audit finds no handled-request entry matching a timed-out comparison attempt, but that evidence alone does not locate where a datagram was lost or delayed.

Separate Recovery Experiment

Run 20261101-194117 stops the background sender in the third phase and adds no link impairment. Verification uses packet counters and rule snapshots rather than accepting the controller log alone as proof of success.

Table 13. Preserved recovery checks

Check	Recorded evidence	Outcome
Alternate path carried requests	108 sync request packets on s1 priority 200, output 6	PASS
Temporary rules were removed	No priority-200 entries on s1, s3, or s4 in phase 3	PASS
Primary path carried traffic again	s1 priority-100 request counter rises from 132 to 204	PASS
Application continues operating	All three clients have successful recovery-phase replies	PASS

The Path A increment is 72 packets between the two snapshots. It verifies resumed use of the primary path, without assigning an exact switching instant. The five optimized comparison runs also contain positive Path B request counters during the loaded phase, ranging from 102 to 111. Recovery is therefore supported by both data-plane traffic and application replies.

Testing and Validation

Final verification checks the source revision described in this report without modifying the recorded archive. The Python suite contains 77 tests and the independent browser-clock suite contains 10 tests. Both pass. Python tests also run in an isolated checkout inside the Linux VM; a fresh comparison, plot generation, recovery run, and post-run environment check pass there on 3 November 2026.

Table 14. Test coverage in the repository

Area	Checks represented in the suite
Packet protocol	35-byte round trip, malformed packets, non-finite timestamps, impossible delay and overflow
Clock calculations	Initial offset, drift, step correction, and gradual slew
Standalone client	Every mismatch field records INVALID; no rejected clock adjustment; next attempt can succeed
UDP integration	Real loopback server exchange and three-cycle step/slew clients
Controller and topology	Fixed ports, queue configuration, threshold response, recovery, and preserved original rules
Evaluation	Timeout accounting, missing delay, convergence rounds, failed-pair diagnostics, and cleanup
CLI and launcher	Menu dispatch, setup failure/reuse, paths with spaces, unsupported OS, wrapping, and partial CSV rows
Browser clock logic	Drift, timestamp correction, asymmetric delay, polling, recovery, reset, invalid inputs, and bounded history

The standalone mismatch regression covers response type, client ID, sequence number, and echoed timestamp. Its tests first reproduced missing CSV rows; the current behavior records INVALID through the existing failure path. This avoids reducing the denominator of a reliability calculation by silently dropping an attempt.

Live Mininet checks require all 30 directed host pairs to be reachable before measurement. A completed run must also contain at least one successful sync sample for every client. Recovery requires all four checks listed on the previous page. A real TCP-listener test distinguishes an occupied port from a recently closed connection in TIME_WAIT, allowing consecutive controller runs.

Import-level test doubles permit some topology and handler tests outside the VM. They do not produce performance evidence: the benchmark rejects non-Linux and non-root execution, checks Mininet and OS-Ken imports, and uses real OVS commands. Passing automated tests also does not prove absence of every defect; final demonstration and individual viva remain separate evaluation activities.

Coverage of the Official Rubric

The official rubric allocates 15 marks to D1 and 25 to D2. The mapping below identifies concrete implementation and evidence for each component. It is a coverage guide, not a claim that marks or viva performance have already been awarded. [1]

Table 15. D1 and D2 criteria mapped to verifiable project evidence

Component	Marks	Evidence and report location
D1 Problem understanding and architecture	2	Objectives and component roles; architecture diagram and host table; pages 4-6.
D1 TCP or UDP implementation	4	UDP packet schema, four timestamps, concurrent server, request validation and failure accounting; pages 7-8.
D1 Mininet and SDN	4	Four-switch diamond, fixed ports, ARP proxy, proactive forwarding and queue setup; pages 6, 9.
D1 Socket-SDN integration	3	UDP 12300 classification and load-triggered bidirectional rerouting; pages 9-10.
D1 Initial testing and demo	2	Real UDP integration tests and 30 directed host-pair checks; pages 11, 15, 19.
D2 Complete application functionality	4	Three persistent clients, reference server, clock correction, and logged outcomes; pages 7-8, 11.
D2 Dynamic SDN functionality	6	Two-second port monitoring, priority-200 rules, and automatic three-poll recovery; pages 10, 14.
D2 Challenging scenario and robustness	4	12M offered load, forward NetEm delay and jitter, failures and validated recovery; pages 11, 13-15.
D2 Performance evaluation and comparison	5	Five completed baseline comparisons, per-phase analysis, failed attempts and convergence; pages 12-14.
D2 Documentation demo and viva	6	This report, README setup, source and configuration, terminal guide and reproducible evidence. Individual understanding is demonstrated during the viva.

The Project 23 spreadsheet additionally asks for timestamp exchange, RTT calculation, offset estimation, multiple clients, prioritization, controlled delay, synchronization-error measurement, and a congestion comparison. Each is implemented and addressed above. The application data plane uses the required UDP transport; TCP is confined to the OpenFlow control connection in the evaluated network. [2]

Challenges and Limitations

A time estimate cannot generally separate clock offset from unequal one-way delay using only one four-timestamp exchange. Persistent forward/reverse asymmetry can therefore leave a biased correction. The added impairment is deliberately directional, making this assumption observable rather than proving that the estimator handles every delay pattern.

Virtual clocks inside one VM provide convenient reference error because namespaces share a system clock. This differs from synchronizing independent physical oscillators. The large artificial drift rates are demonstration inputs. No hardware timestamping, kernel clock discipline, authentication, or full NTP behavior is implemented. Float timestamp precision and VM scheduling also limit conclusions about absolute timing accuracy.

The optimized policy changes queue treatment and routing together, and its alternate path has greater configured capacity. The experiment therefore cannot isolate which change produces each gain. Only five complete comparisons are available; there are no randomized-order trials, confidence intervals, broad client-count scaling tests, or physical-network measurements.

A timeout is an application event. The tests do not capture every packet to establish a loss location or packet-loss rate. A rejected or late reply can also affect a later attempt because the client accepts or rejects its first received response rather than draining unrelated datagrams until a fixed deadline. Request matching prevents incorrect adjustment but does not provide retransmission or guaranteed delivery.

The controller uses a fixed host map, fixed paths, one monitored uplink, and one load threshold. It is not a general topology-discovery or link-failure recovery system. The three-low-poll rule dampens a return to the primary path, but no exact failover-latency claim is supported by the saved snapshots.

Future Scope

- Add a receive-until-deadline loop, explicit retry policy, and better handling of late replies while preserving one final CSV result per attempt.
- Run separate QoS-only and routing-only experiments on equal-capacity paths, with randomized repetition and uncertainty estimates.
- Evaluate larger client populations and independent hosts; retain equivalent offered load and resource measurements.
- Add discovery, stale-datapath handling, configurable paths, and direct failure detection before claiming general network recovery.
- Explore authenticated timestamps and operating-system clock discipline as separate extensions, with appropriate privileges and safety controls.

Conclusion

Network Time Sync implements the assigned multi-client UDP timestamp exchange and connects it to measurable SDN behavior in Mininet. The application protocol remains the same while the controller changes queue treatment and forwarding. The preserved five-run comparison supports lower mean clock error, RTT, and failed-attempt percentage under the combined optimized policy. The separate recovery test confirms that traffic can return to the primary route after background load stops.

The evidence also sets clear limits. Initial convergence is slower in three optimized runs, timing samples are taken after attempts, and the alternate path has a different capacity. The project is therefore best understood as a controlled evaluation of a complete SDN policy for virtual-clock synchronization. Its source, fixed configuration, archived measurements, tests, and execution instructions allow an evaluator to reproduce and explain the behavior.

References

[1] Official SDN-Socket Programming Guidelines and Rubrics, pages 1-2. Repository: docs/Reference docs/SDN-Socket Programming-Guidelines_Rubrics.pdf.

[2] Official project catalogue, Project 23, and Section K team allocation, Team 11. Repository: docs/Reference docs/SDN_Socket_Programming_30_Projects.xlsx and Sec K Batch_Random_Team_Project_Allocation.xlsx.

[3] Network-Time-Sync source, final branch, revision 55d1f576e8dc. app, sdn, topology, evaluation, scripts, tests and web. https://github.com/Annadata-Aniketh/Network-Time-Sync/tree/55d1f576e8dcc93b682d0d4e6b16bbcdd2641554

[4] Recorded VM evidence. evaluation/reference/vm-results.tar.gz, audit.json and environment.txt. Comparisons dated 1 November 2026; original recorded source revision 06562b2.

[5] D. Mills, J. Martin, J. Burbank and W. Kasch. RFC 5905, Network Time Protocol Version 4, June 2010. Four-timestamp estimation background. https://datatracker.ietf.org/doc/html/rfc5905

[6] Python Software Foundation. Python 3.12 socket documentation. https://docs.python.org/3.12/library/socket.html

[7] Mininet Project. Mininet Overview. https://mininet.org/overview/

[8] OpenStack. OS-Ken documentation. General controller APIs; the tested package version is stated separately. https://docs.openstack.org/os-ken/latest/

[9] Open vSwitch Project. Quality of Service FAQ. https://docs.openvswitch.org/en/latest/faq/qos/

Appendix Setup and Execution

Use Ubuntu 24.04 or Linux Mint 22 with Python 3.12, Git, internet access for initial installation, and sudo. Start from the repository root after cloning the final branch. The launcher installs or reuses the project environment and opens the selection menu.

```
git clone --branch final \
  https://github.com/Annadata-Aniketh/Network-Time-Sync.git
cd Network-Time-Sync
./timesync
```

Reproduce the Measurements

```
./timesync doctor
./timesync --batch --scenario both --duration 60
./timesync --batch --scenario recovery --duration 60
./timesync test
node --test tests/clock-model.test.cjs
```

The Node.js command tests only the browser clock logic and is optional for running the Linux experiment. To generate plots, select menu option 5 and choose the completed comparison directory. Option 6 prints its summary. Option 9 runs the separate recovery check. The default 60 seconds per mode is retained for the ten-minute demonstration.

Review and Transfer Evidence

```
.venv/bin/python -m evaluation.audit_results
sudo bash scripts/export_results.sh
```

The audit recalculates the preserved reference archive into ignored evaluation/runs/audit outputs. The export script bundles local VM runs and environment information; it does not replace the reference archive. The reference SHA-256 is:

```
3b094b35eebdcef4f8252b8a915cc629d7100e24c7aed5993e95093e7cade2a2
```

Practical Demo Sequence

Use i to explain the exchange, then 4 for the comparison. Show plots and the summary with 5 and 6, then use 9 to verify recovery. During the viva, explain why two-way delay and clock error differ, why source and destination port matches are both needed, and why flow counters are stronger rerouting evidence than a timing improvement alone.

Run one network experiment at a time. Ctrl+C invokes cleanup. If a previous run crashed, inspect its logs and use sudo mn -c before retrying. Do not run that cleanup while another Mininet experiment is active. Fill the faculty guide, academic year, and submission-date placeholders on the cover before submission.